A Tutorial on using the BlackBox Debugger with Catalina

INTRODUCTION....2

What is BlackBox?....2
What is the status of BlackBox?....2
What are the prerequisites for BlackBox?....2
Why is BlackBox "unorthodox"?....3
What is the difference between BlackCat and BlackBox?....3
Are there any known issues with BlackBox?....3
Who should I contact about BlackBox?....3

PREPARING PROGRAMS FOR USE WITH BLACKBOX....4

Overview....4
Preparing programs using Catalina from the command line....5
Preparing programs using Catalina from Code::Blocks....6
Loading programs into the Propeller....7
From the command line....7
From Code::Blocks....7
Starting BlackBox from the command line....7

A DEMONSTRATION OF BLACKBOX....8

The example program....8
Compiling and Loading the example....8
Compiling and Loading on the DracBlade, Demo Board or TriBladeProp....8
Compiling and Loading on the RamBlade....9
Compiling and Loading on the Hydra or Hybrid....10
Compiling and Loading on Morpheus....10
Compiling and Loading on the C3....11
Starting BlackBox to debug the example....11
Displaying source files....12
Single Stepping through the program....13
User Breakpoints....16
Printing and Updating Variables....17
Combining User and Virtual Breakpoints....18

WHAT TO DO NEXT?....20

KNOWN ISSUES WITH BLACKBOX....20

Copyright 2013 Ross Higson

Introduction

This document provides a brief tutorial to get you up and running with the BlackBox debugger for the Catalina C compiler. It assumes a reasonable degree of familiarity with both the Propeller and with Catalina.

A full reference manual for **BlackBox Reference Manual** is also provided with Catalina, containing more detail about various BlackBox commands and features. However, it is not necessary to read the reference manual before starting this tutorial.

For the sake of brevity, this document assumes you are using both Catalina and BlackBox under Windows. If you are using Linux, the commands and command options are identical, but some file and directory names may have to be changed

What is BlackBox?

BlackBox is a command line application that allows source level debugging of C programs that have been compiled using the Catalina C compiler, and which are loaded and executing on a supported Propeller platform.

BlackBox runs on the PC, and communicates via a serial connection with the program being debugged.

BlackBox provides the ability to display C source files, set and clear user breakpoints, single step through C programs (BlackBox uses a slightly unorthodox method of doing this), and display or modify the values of any local or global C variables.

What is the status of BlackBox?

BlackBox is capable of debugging arbitrary C programs, including all the demonstration programs provided with Catalina.

BlackBox supports Propeller 1 and Propeller 2 programs.

BlackBox supports debugging of CMM, LMM and XMM programs on all Propeller 1 platforms supported by Catalina (with the exception of XMM programs executed from SPI Flash – such programs must be recompiled to use SPI RAM only.

BlackBox supports debugging of CMM, LMM & NMM programs on all Propeller 2 platforms supported by Catalina.

What are the prerequisites for BlackBox?

BlackBox has been tested under Windows XP SP2 (32 bit) and Windows 7 (64 bit), but should also run on other versions of Windows.

BlackBox has been tested under Linux Fedora, Debian and Ubuntu distribution. It should also run under other Linux distributions.

Programs to be debugged using the current release of BlackBox must be compiled using Catalina release 4.0 or greater – the current version of BlackBox may not work with any earlier versions of Catalina (instead, use the version that came with the version of Catalina you are using). Programs must be prepared by compiling them using a new Catalina **-g** or **–g3** command line options (the meaning and use of these options is described later in this document).

BlackBox requires that the program being debugged has a cog free to implement the BlackCat serial protocol (BlackBox and BlackCat use the same protocol). Other than that, the overhead of BlackBox on the compiled program is limited to the execution of approximately 100 additional instructions (400 bytes) during program initialization. When not actually stopped at a breakpoint, the program executes as usual (i.e. at normal speed) and is unaware of BlackBox – this makes BlackBox suitable for debugging real-time or time-critical programs.

BlackBox also requires a USB or RS232 serial connection to the Propeller running the program to be debugged. On Propeller 1 platforms that use the HX512 SRAM expansion card (e.g. the Hydra and the Hybrid), a special cable is required – this is the same cable as is used for the Catalina Payload program loader – see the **Catalina Reference Manual** for more detail. On Propeller 2 platforms a PropPlug can be used on pins 50 & 52.

Why is BlackBox "unorthodox"?

The BlackBox debugger uses a slightly unusual approach to breakpoints, which can be of two types – user or virtual. This can sometimes lead to unexpected results when a program is executed using a combination of manual breakpoints, and the various single step operations (**step next**, **step into**, **step out**) which use virtual breakpoints.

This is more fully explained later in this document.

What is the difference between BlackCat and BlackBox?

BlackCat and BlackBox are both source level debuggers for Catalina. Functionally, the two programs are quite similar, and both were developed (independently) based on work done co-operatively by Ross Higson and Bob Anderson. However:

- BlackBox is a source level debugger with a command line interface, and runs under Windows and Linux.
- BlackCat is a source level debugger with a graphical user interface, and it currently runs under Windows only.

Are there any known issues with BlackBox?

There are no known problems, but there are a few things not yet implemented – see the section **Known Issues** later in this document.

This is particularly important if you are using BlackBox under Linux, as there is an issue concerning serial communications between BlackBox and the Propeller that can affect Linux platforms.

Who should I contact about BlackBox?

Contact Ross Higson at ross@thevastydeep.com

Preparing programs for use with BlackBox

Overview

Preparing a C program can be as simple as including the **–g** or **–g3** command line options when compiling the program with Catalina. This instructs Catalina to do two things:

1. Generate additional information that is required by the BlackBox debugger. A **.debug** file is generated for each C source file, and these are then combined into one **.dbg** file that can be loaded by BlackBox (the **.debug** files can then be deleted).
2. Select a BlackCat/BlackBox enabled target for the program. Suitable targets are provided for CMM, LMM and XMM programs. These targets contain additional initialization code that loads the BlackCat debug cog, and performs other additional tasks required to prepare the kernel for co-operation with the debug cog.

In addition, the **–g3** option does the following:

3. Disable some compiler optimizations that can make debugging more complex. For example, this option ensures that all functions have a stack frame whether they need it or not, which simplifies single stepping programs. While using this option can increase program size and execution time slightly, it is recommended, unless it makes the program too large to load, or execute too slowly.

There are a few things that you need to be aware of when preparing programs for use with BlackBox:

- The program must have a spare cog available (to run the BlackCat debug cog).
- The program must have two pins (normally Propeller pins 30 and 31, but others can be used) available for serial communication with the BlackBox debugger running on the host.
- You cannot specify your own target – to use BlackBox you must let Catalina select a BlackBox-enabled target.
- The resulting program executable will be slightly larger – about 400 additional bytes of initialization code, plus up to 512 bytes of debug cog code (however, the space used for the debug cog code is reclaimed during initialization for use as program stack or heap space once the program is executing).

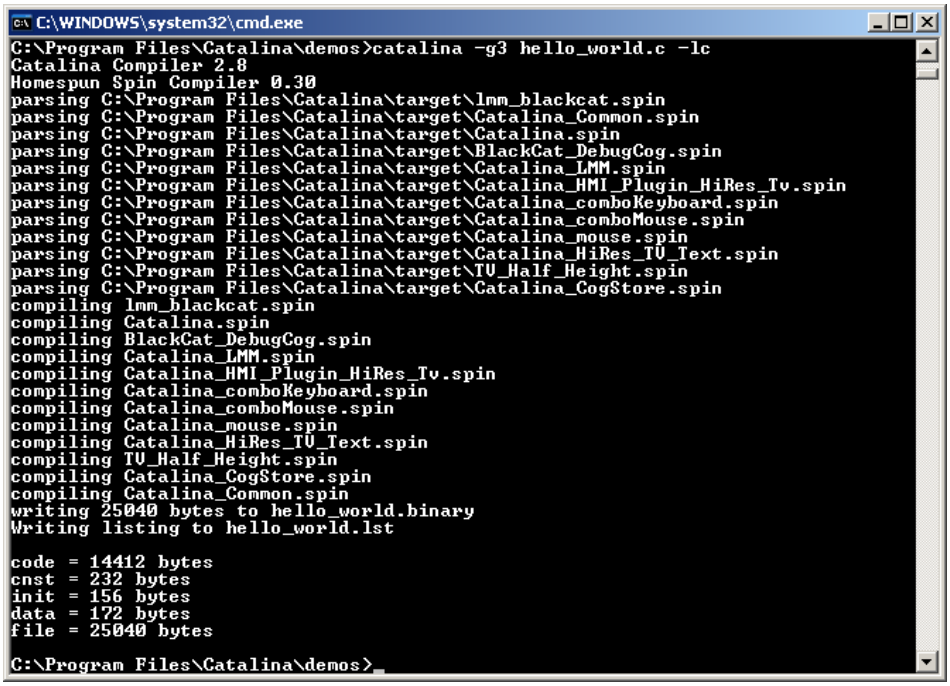
Copyright 2013 Ross Higson

Preparing programs using Catalina from the command line

As mentioned above, simply include the **–g** or **–g3** command line options along with all the other normal options when compiling the program. For example, after setting up to use Catalina (using the **use_catalina** batch file) execute the following command in the Catalina demo directory:

```
catalina -g3 hello_world.c -lc
```

This will produce output similar to the following:



```
C:\WINDOWS\system32\cmd.exe
C:\Program Files\Catalina\demos>catalina -g3 hello_world.c -lc
Catalina Compiler 2.8
Homespun Spin Compiler 0.30
parsing C:\Program Files\Catalina\target\lmm_blackcat.spin
parsing C:\Program Files\Catalina\target\Catalina_Common.spin
parsing C:\Program Files\Catalina\target\Catalina.spin
parsing C:\Program Files\Catalina\target\BlackCat_DebugCog.spin
parsing C:\Program Files\Catalina\target\Catalina_LMM.spin
parsing C:\Program Files\Catalina\target\Catalina_HMI_Plugin_HiRes_Tv.spin
parsing C:\Program Files\Catalina\target\Catalina_comboKeyboard.spin
parsing C:\Program Files\Catalina\target\Catalina_comboMouse.spin
parsing C:\Program Files\Catalina\target\Catalina_mouse.spin
parsing C:\Program Files\Catalina\target\Catalina_HiRes_TV_Text.spin
parsing C:\Program Files\Catalina\target\TV_Half_Height.spin
parsing C:\Program Files\Catalina\target\Catalina_CogStore.spin
compiling lmm_blackcat.spin
compiling Catalina.spin
compiling BlackCat_DebugCog.spin
compiling Catalina_LMM.spin
compiling Catalina_HMI_Plugin_HiRes_Tv.spin
compiling Catalina_comboKeyboard.spin
compiling Catalina_comboMouse.spin
compiling Catalina_mouse.spin
compiling Catalina_HiRes_TV_Text.spin
compiling TV_Half_Height.spin
compiling Catalina_CogStore.spin
compiling Catalina_Common.spin
writing 25040 bytes to hello_world.binary
Writing listing to hello_world.lst

code = 14412 bytes
cnst = 232 bytes
init = 156 bytes
data = 172 bytes
file = 25040 bytes

C:\Program Files\Catalina\demos>_
```

Note that Catalina has automatically used the **lmm_blackcat.spin** target instead of the more usual **lmm_default.spin** target (both **BlackCat** and **BlackBox** share the same target).

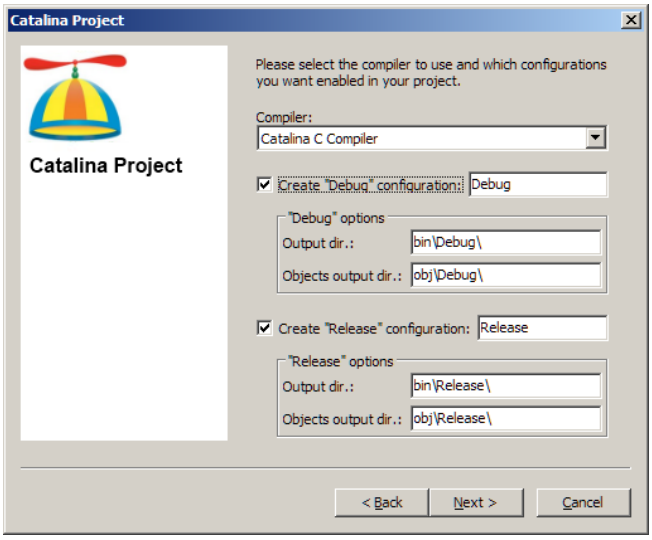
In addition to the normal **hello_world.binary**, Catalina will also generate the following files:

- A file called **hello_world.lst** (a listing file is *always* produced when the **–g** or **-g3** command-line options are used).
- A file called **hello_world.debug** (one such file is generated file for each C source file, named with the same name as the C source file but with a **.debug** extension)
- A file called **hello_world.dbg** (one such file is generated for the whole compilation, named with the same name as the output binary file but with a **.dbg** extension).

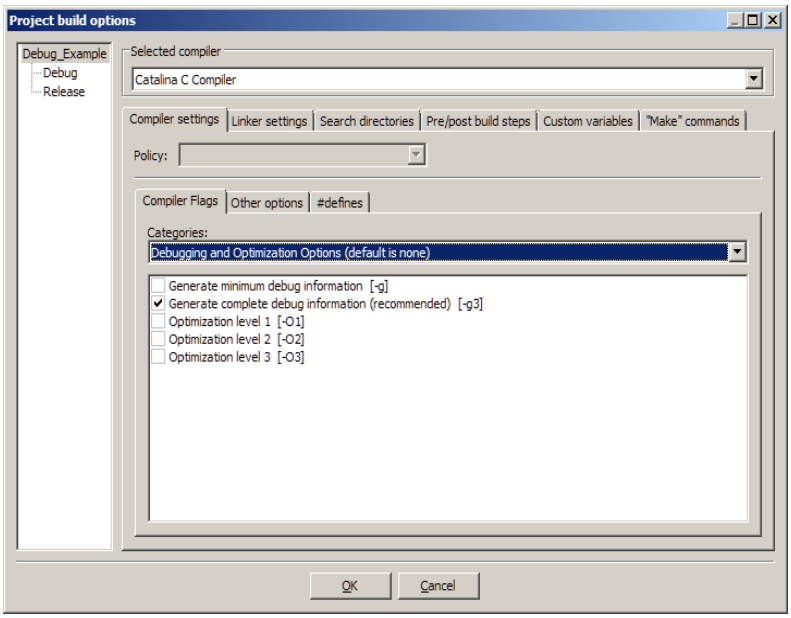
Note that the files actually required for debugging are the **hello_world.binary** (or **hello_world.bin** on the Propeller 2) and the **hello_world.dbg** file. The other files are left for reference only, and can be deleted once the compilation is complete. However, also note that it is *essential* that the correct dbg is used when debugging – this file contains information about the location in the binary file of various program elements. If the program is modified and recompiled, the .dbg file *must* be regenerated

Preparing programs using Catalina from Code::Blocks

Catalina release 3.1 provides a **Code::Blocks** Project Wizard that can be used for creating new Catalina projects. This wizard will allow you to set up new projects to build both a **Debug** or a **Release** version of your program. The Debug version will have debugging enabled automatically. Make sure you select the **Create "Debug" configuration** option when you create your Catalina project:



For existing projects, you can to tell **Code::Blocks** specifically how to a enable debugging versions of your program. Select **Projects->Build Options** and then the Compiler Settings tab. Select the **Debugging and Optimization Options** category, and select the desired debugging option:



Then you simply compile and link your program as normal.

The **.debug** files generated by Catalina will be placed in the same directory as the source files, but the **.lst** and **.dbg** files will be placed in the same directory as the binary output file.

Loading programs into the Propeller

From the command line

There are various methods for loading programs into the Propeller, including the following:

1. Using the Catalina Payload program loader. This program supports CMM, LMM and XMM programs on the Propeller 1, and CMM, LMM and NMM programs on the Propeller 2. Refer to the **Catalina Reference Manual** for more details.
2. Using the Parallax Propeller tool (CMM and LMM programs on the Propeller 1 only).
3. For EMM programs, program them into the EEPROM of the Propeller.
4. For SMM programs, write them onto an SD Card and load them via Catalyst.
5. For CMM, LMM or XMM programs, write them to an SD card and then use the Catalina Generic SD Program Loader (for multi-CPU systems such as the TriBladeProp or Morpheus, you may also need to use the Catalina Generic SIO Program Loader)

For example, to automatically locate the first port with a Propeller attached, and then load a file **example.binary**, the following payload command might be used:

```
payload example
```

In any case, all programs built using the **–g** or **–g3** options always stop after the execution of the initialization code, and wait till the BlackBox debugger establishes serial communications with the BlackCat debug cog.

From Code::Blocks

there are several menu items that can be used to download a program and start the debugger. From the **Tools** menu, select:

Download to Hub RAM and debug

Download to XMM RAM and debug

Both of these commands will download the program just compiled to the Propeller, and then start the Blackbox debugger in a command window.

Starting BlackBox from the command line

Start BlackBox to debug a program (in this case **example.binary**) like can be done by entering a command similar to the following:

```
blackbox example
```

This command accepts parameters described in the **BlackBox Reference Manual**.

BlackBox should automatically determine the port to use for debugging – even on the Hydra or Hybrid platforms where a special debug cable is required. If it does not, the port to use can be specified via the **-p** option. For example:

```
blackbox -p2 example
```

Note that in BlackBox, the **-p** option specifies the port. It is not necessary to specify whether a Propeller 1 or a Propeller 2 is in use – BlackBox detects this automatically.

Note that under Linux, starting BlackBox automatically raises DTR, which can cause the Propeller to reset. If this occurs on your platform, it is necessary to temporarily disable the RESET connection – see the **Known Issues** section for more details.

A Demonstration of BlackBox

This section provides a real-world demonstration of BlackBox. It uses an example program provided with the Catalina distribution, in the **demo\debug** subdirectory.

The example program

The example program requires an external VGA or TV display and a keyboard, but no mouse or floating point libraries.

The example program consists of the following C source files:

```
debug_test.h
debug_main.c
debug_functions_1.c
debug_functions_2.c
```

The example program performs various test functions designed to explore the capabilities of BlackBox. Most of the functions print their results on the display, and some require user interaction via the keyboard.

Compiling and Loading the example

To compile the example program, open a command window and set up Catalina for use as normal (i.e. by executing the **use_catalina** script). Then go to the **demo\ debug** directory.

Specific notes for compiling and loading the program on various platforms are given below. Batch files are provided for building the program, and a **Code::Blocks** project is also provided, which by default builds for the **TRIBLADEPROP CPU_1** – if you plan to use Code::Blocks you will need to edit this configuration to suit your platform.

Note that if you use the batch file to compile the program, the binary (and corresponding lst and dbg files) will be located in the **demos\debug** directory, whereas if you use **Code::Blocks** the binary, lst and dbg files will end up in the **demos\debug\codeblocks\bin\Debug** directory.

Compiling the program will generate various files similar to the following:

```
example.binary
example.dbg
example.lst
debug_main.debug
debug_functions1.debug
debug_functions2.debug
```

In this demonstration, we will use the binary file and the dbg file. The others are generated for information purposes whenever the **–g** or **–g3** option is specified (in this case the **–g3** option is included by the scripts that we will use to build the example program).

Compiling and Loading on the DracBlade, Demo Board or TriBladeProp

If you have a DracBlade, Demo board or TriBladeProp, then you can build the program using the command **build_all** in the demo\debug directory, specifying the Propeller platform as a command line parameter.

If you have a TriBladeProp, you must also specify the CPU – the demo program requires both a screen and keyboard device, so is most easily built for CPU #1. For example:

```
build_all DRACBLADE
```

or:

```
build_all DEMO
```

or:

```
build_all TRIBLADEPROP CPU_1
```

Load the resulting **example.binary** file into the Propeller using the Catalina Payload program loader:

```
payload -pN example
```

Where ***N*** is the port connected via a normal RS232 or USB cable to pins 30 and 31 on the Propeller (on the TriBladeProp, make sure the jumpers and cables are set to connect to CPU #1).

When the binary has been loaded, all the drivers are initialized, so you should see a blank display appear. However, the C program will not begin to execute until we start BlackBox.

Compiling and Loading on the RamBlade

If you have a RamBlade, then you can build the program using the command **build_all** in the demo\debug directory, specifying RAMBLADE as a command line parameter:

```
build_all RAMBLADE
```

Load the resulting **example.binary** file into the Propeller using the Catalina Payload program loader:

```
payload -pN example
```

Where ***N*** is the port connected via a USB cable to pins 22 and 23 on the Propeller (on the TriBladeProp, make sure the jumpers and cables are set to connect to CPU #1).

When the binary has been loaded, all the drivers are initialized, so you should see a blank display appear. However, the C program will not begin to execute until we start BlackBox.

Note that on the RamBlade, you will need a second USB cable connected to pins 30 and 31 to see the output of the program.

*NOTE: In **Catalina_Common.spin**, the RamBlade platform is set up to use pins 22 & 23 for BlackBox communications. This allows the debugging of LMM programs, including those that use pins 30&31 for serial communications – but this configuration cannot be used to debug XMM programs since these programs need pins 30 & 31 to communicate with the XMM RAM. To debug XMM programs requires that the RAMBLADE configuration in **Catalina_Common.spin** be modified so that pins 22 & 23 are used for BlackBox – however, this means such programs cannot also use serial I/O, since pins 30&31 are used for SRAM access, and pins 22 & 23 are being used for BlackBox.*

Compiling and Loading on the Hydra or Hybrid

If you have a Hydra and Hybrid (with or without the HX512 SRAM card) you will need a special cable plugged into the Propeller mouse port, and you must also build the demo program to exclude the normal mouse driver. For example, use the **build_all** command as follows:

```
build_all HYBRID NO_MOUSE
```

or:

```
build_all HYDRA NO_MOUSE
```

The special cable you will need is described in the **Catalina Reference Manual**. A normal serial cable is used to load the program, but the default debugging target for the Hybrid and Hydra assumes that the mouse port will be used for debugging programs.

Load the **example.binary** file into the Propeller using the Catalina Payload program loader:

```
payload -pN example
```

Where ***N*** is the port connected using a normal RS232 or USB cable to pins 30 and 31 on the Propeller.

Compiling and Loading on Morpheus

If you have a Morpheus, the situation is a little more complex because neither of the Morpheus CPUs normally have simultaneous access to both a keyboard and display. This means proxy drivers must be used. A batch file called **build_ morpheus** is provided specifically to build the program and the required proxy drivers for Morpheus. It requires no parameters, so just enter:

```
build_morpheus
```

This batch file builds several programs:

example_client_1.binary	the demo program built to run on CPU #1;
example_server_2.binary	a proxy server built to run on CPU #2;
null.binary	a dummy program (used as described below).

A normal serial or USB cable must be connected to both CPU #1 and CPU #2 (or the one cable can be swapped between loads).

Load the programs in the following order:

```
payload -pN null
payload -pM example_server_2
payload -pN example_client_1
```

Where ***N*** is the port connected via an RS232 serial or USB cable to pins 30 and 31 of CPU #1, and ***M*** is the port connected via a USB cable to pins 30 and 31 on CPU #2.

The null program is used to ensure that CPU #1 is not executing a program that interferes with our ability to load CPU #2 – on Morpheus, the connections between the two Propellers makes this possible.

NOTE: Using the correct port for each load is obviously critical, as is the order of loading the programs, and it is also important to load the null program before any other load. Things can get even more confusing if you happen to be using COM port 1 to talk to CPU #2 and COM port 2 to talk to CPU #1!

Compiling and Loading on the C3

If you have a C3, then you can build the program using the command **build_all** in the demo\debug directory, specifying the C3 as a command line parameter. You can also specify the HMI option to use (e.g. VGA) if you do not want the default (which is to use the TV output).

For example:

```
build_all C3
```

or:

```
build_all C3 HIRES_VGA
```

Load the resulting **example.binary** file into the Propeller using the Catalina Payload program loader:

```
payload -pN example
```

Where ***N*** is the port connected via a normal RS232 or USB cable to pins 30 and 31 on the Propeller (on the TriBladeProp, make sure the jumpers and cables are set to connect to CPU #1).

When the binary has been loaded, all the drivers are initialized, so you should see a blank display appear. However, the C program will not begin to execute until we start BlackBox.

Starting BlackBox to debug the example

Start BlackBox using the command:

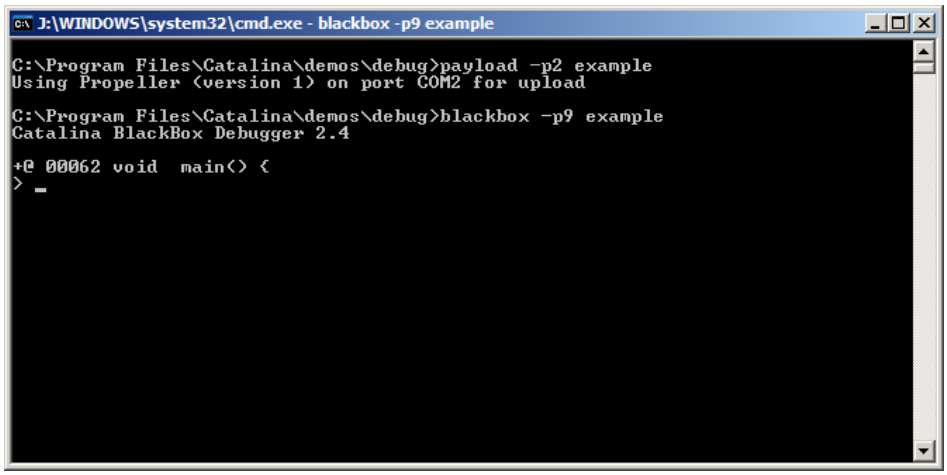
```
blackbox -pN example
```

or (on Morpheus only):

```
blackbox -pN example_client_1
```

Where ***N*** is the port connected to the Propeller mouse port (Hybrid or Hydra only), or to pins 30 and 31 of the appropriate Propeller CPU (other platforms).

For example, here is how your command window might look after using **payload**, and starting **blackcat,** if you were using a Hybrid, with your normal propeller connected to port 2, and the special cable connected to port 9:



BlackBox is now waiting for commands, and indicating that the program is stopped at line **62** – which is the first line of the function **main**.

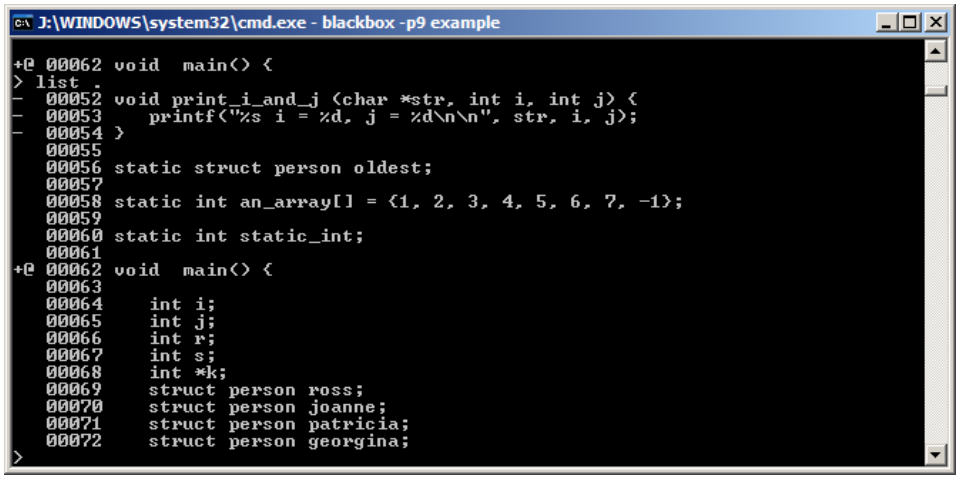
The **+** at the start of the line indicates that a virtual breakpoint has been placed on this line, and the **@** indicates this is the source line at which the debugger is currently stopped.

Displaying source files

The first thing we will do is display the source around the point we are currently stopped. To do this, we use the **List** command, specifying the parameter '**.**' which refers to the current debug location. Enter the following command (note there is a space between the '**list**' and the '**.**'):

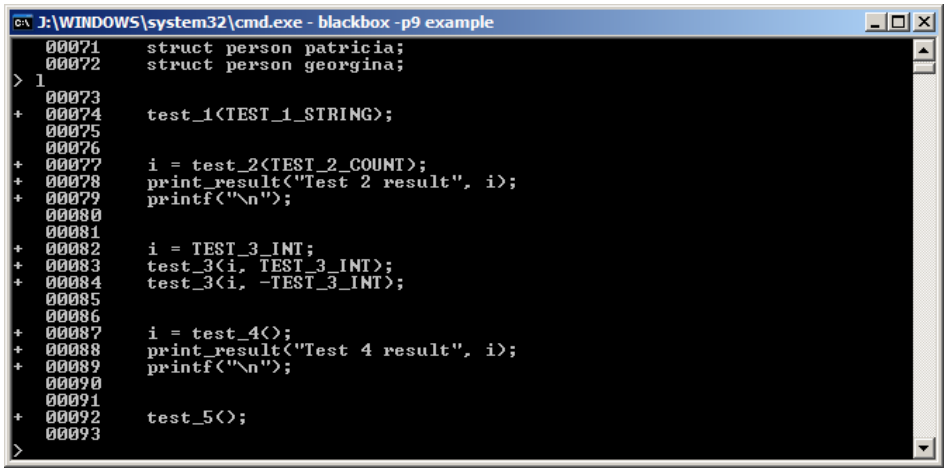
```
list .
```

The result should look similar to the following:



The source lines 52 to 72 are displayed (i.e. 20 lines centered on the current debug location). The **–** at the beginning of lines 52 to 54 indicate that there is code associated with these lines. Other than line 62, all the other lines are type and variable declarations, and have no actual code associated with them. This is significant because only lines with code can have a breakpoint set on them.

To continue looking at the source, use the '**list**' command again, but this time with no parameter (and we can abbreviate it to just '**l**'). The result should now look similar to the following:



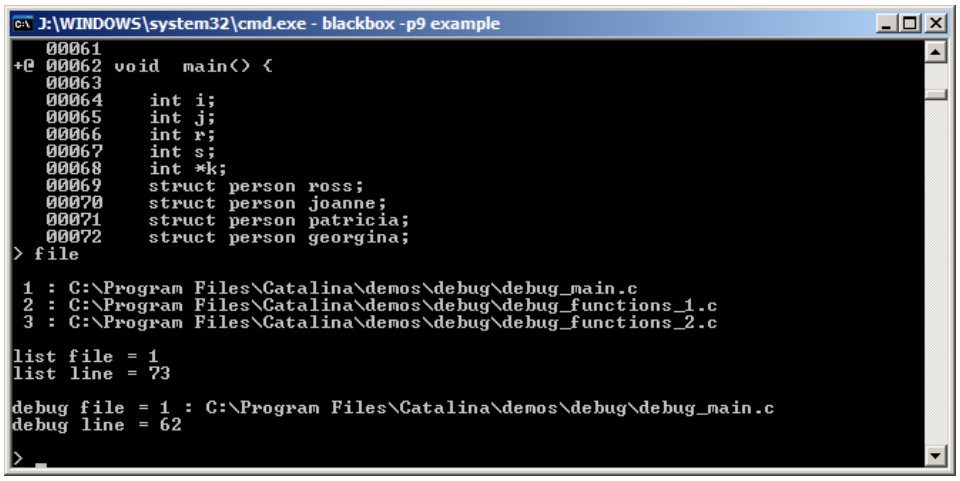
You can continue listing this source, or list a specific line or function. For example, try any of the following:

```
list test_1
list 50
list 20-30
```

You can also list source in other files. To see the list of source files that comprise the current program, use the **File** command. Enter the command:

```
file
```

The result should look similar to the following:



This shows the three C source files that comprise the current program (the debugger does not know about **.h** files, only **.c** files), and also indicates the current list line and the current debug line. The File command can also be used to change files, so that you can list lines in a different file.

For example, try:

```
file 3
list 50
```

No matter which file you are currently listing, the command '**file .**' will always set the current list file and line to be the current debug location, and the command '**list .**' will always list the current debug location.

In the remainder of this tutorial, we will always show all commands entered in full – but all commands can be abbreviated, usually down to a single character. For example, the following are all equivalent:

```
list 50
lis 50
li 50
l 50
```

Single Stepping through the program

Now we'll try executing a line or two of the program. There are two different commands that can be used to execute the current line of code:

Step Next execute the current line of code and step to the next line in the current function that contains executable code. Can also be abbreviated to either **Step** or **Next**.

Step Into execute the current line of code, but if there is a function call in this line, step *into* that function and stop at the function entry point. Otherwise, this command also just steps to the next line in the current function. Can also be abbreviated to **Into**.

When you are single stepping *within* a function, there are various ways to get *out* again:

Step Out execute lines until the program calls another function, or reaches any existing breakpoint. The effect of this is that if the current function was reached via 'Step Into' then this will return to the calling function. Can also be abbreviated to just **Out**.

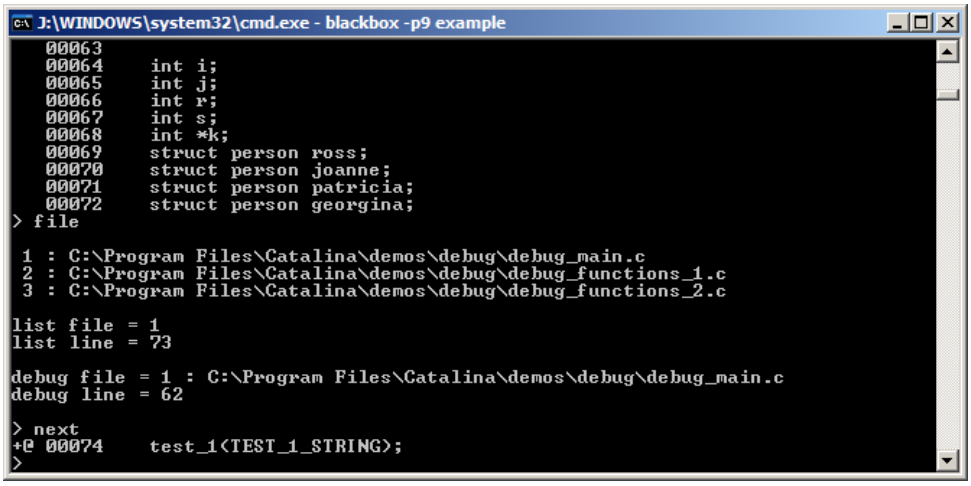
Step External execute lines until the program reaches *any* line of code *external to the current function*. This may be the next line in the function that called the current function, or a line in a function that the current function calls.

Step Up execute lines until the program returns from the current function. Actually, this returns to the call prior to the function call where the current stack frame was created. This will always be the calling function if the **–g3** command line option was used. If the **–g** command line option was used instead, this may cause the program to return to a function call further up calling hierarchy, not the immediately preceding function call.

To single step one line, enter the **Next** command:

```
next
```

The result should look similar to the following, indicating we have stepped to the next line containing executable code, which is line 74:



. This line contains a function call, so we enter the Step Into command:

```
into
```

The result will show we have stepped into the function test_1:

```
J:\WINDOWS\system32\cmd.exe - blackbox -p9 example
   00065     int j;
   00066     int r;
   00067     int s;
   00068     int *k;
   00069     struct person ross;
   00070     struct person joanne;
   00071     struct person patricia;
   00072     struct person georgina;
> file

 1 : C:\Program Files\Catalina\demos\debug\debug_main.c
 2 : C:\Program Files\Catalina\demos\debug\debug_functions_1.c
 3 : C:\Program Files\Catalina\demos\debug\debug_functions_2.c

list file = 1
list line = 73

debug file = 1 : C:\Program Files\Catalina\demos\debug\debug_main.c
debug line = 62

> next
+@ 00074     test_1(TEST_1_STRING);
> into
+@ 00005 void test_1(char *str) {
>
```

At this point we have not actually started executing the function – we are stopped at the entry point to it. However, this function doesn't do anything interesting, so now that we know how to get into a function, let's just get out again. There are several commands we could use, but in this case the **Out** command is the simplest. So enter the command:

```
out
```

The result will show we have stepped back to line 77 of the **main** function:

```
J:\WINDOWS\system32\cmd.exe - blackbox -p9 example
   00067     int s;
   00068     int *k;
   00069     struct person ross;
   00070     struct person joanne;
   00071     struct person patricia;
   00072     struct person georgina;
> file

 1 : C:\Program Files\Catalina\demos\debug\debug_main.c
 2 : C:\Program Files\Catalina\demos\debug\debug_functions_1.c
 3 : C:\Program Files\Catalina\demos\debug\debug_functions_2.c

list file = 1
list line = 73

debug file = 1 : C:\Program Files\Catalina\demos\debug\debug_main.c
debug line = 62

> next
+@ 00074     test_1(TEST_1_STRING);
> into
+@ 00005 void test_1(char *str) {
> out
+@ 00077     i = test_2(TEST_2_COUNT);
>
```

The result of the **test_1** function we just exited should also appear on the display connected to the Propeller.

We can single step through more functions, but it is often useful when debugging to be able to go straight to a particular line of code. To do this we use user breakpoints. This is the subject of the next section.

User Breakpoints

Sometimes, we just want to set a user breakpoint on a specific line and let the program execute until it reaches it.

There are several commands for dealing with user breakpoints:

Breakpoint add a user breakpoint to a specific line. A function name can also be specified to add a breakpoint to the first line of a function. If no parameters are specified this command lists all current user breakpoints.

Delete remove a user breakpoint from a specified line.

Continue Let the program execute until it reaches a user breakpoint.

Go Same as Continue.

In this case, we will set a breakpoint on line 135, and then let the program continue execution until it reaches this user breakpoint.

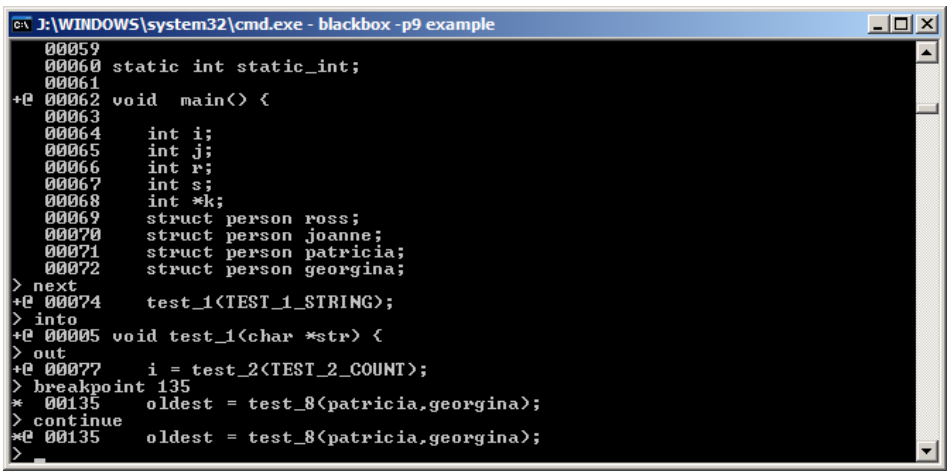
To do that, enter the commands:

```
breakpoint 135
continue
```

Note that the example program we are using requires keyboard input before it will reach line 135 - after entering the **continue** command, at least two keys must be pressed on the keyboard connected to the Propeller to make the program stop again (any two keys will do).

Note also that there may be a pause after entering the **continue** command while the program removes the virtual breakpoints it has previously placed (to support single stepping) – this is characteristic of BlackBox, and can make it appear to take several seconds to respond to some commands.

When the program has stopped at line 135, the result will appear as follows:



In the next section, we will display and modify some program variables.

Printing and Updating Variables

A powerful feature of BlackBox is its ability to display and modify local variables.

To do this we use the Print and Update commands:

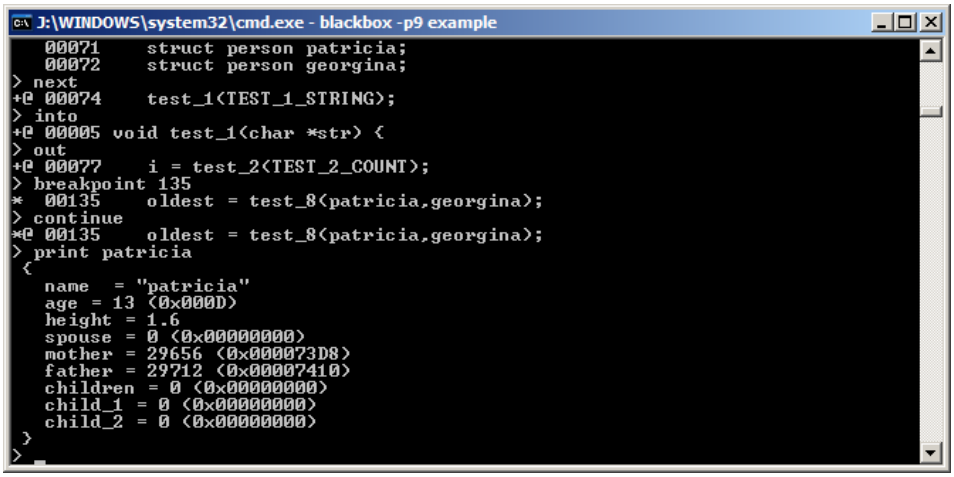
Print This command accepts the name of a variable, and prints its value. The address of the variable can also be printed by prefixing it with **&**, and (if the variable is a pointer) the memory it points to can also be printed by prefixing it with *. If the variable is a structure, each field of the structure will be printed. If the variable is an array, an array index can be included – which can be an integer variable.

Update This command can be used to update the value of a simple scalar variable (or a field of a structure, or an element of an array).

Line 135 calls the function **test_8**, which compares the value of two local variables – called **patricia** and **georgina**. To display the value of **patricia**, enter the command:

```
print patricia
```

The result will look something like:



In this case, the variable is a structure, so all the fields are printed. To print individual fields, try the following commands:

```
print georgina.name
print georgina.height
print georgina.name[2]
```

The results will look something like:

```
J:\WINDOWS\system32\cmd.exe - blackbox -p9 example
*@ 00135    oldest = test_8(patricia,georgina);
> print patricia
 {
   name   = "patricia"
   age  = 13 (0x000D)
   height  = 1.6
   spouse  = 0 (0x00000000)
   mother  = 29656 (0x000073D8)
   father  = 29712 (0x00007410)
   children  = 0 (0x00000000)
   child_1  = 0 (0x00000000)
   child_2  = 0 (0x00000000)
 }
> print georgina.name
 = "georgina"

> print georgina.height
= 1.58

> print georgina.name[2]
= 'o' (0x6F)

> print &georgina
frame offset -172
>
```

The last one is a bit unusual – in this case, BlackBox is telling us that the address of **georgina** is in the current stack frame, at offset -172. Variables may be in registers, in the stack frame or at an absolute location in memory. It makes no difference when printing or updating them – BlackBox knows how to do all these things.

To update the values, enter the following commands:

```
update patricia.height = 1.9
update patricia.age = 14
update patricia.name[3] = char '\0'
```

The results will look something like:

```
J:\WINDOWS\system32\cmd.exe - blackbox -p9 example

> print &georgina
frame offset -172
> update patricia.height = 1.9
= 1.9

> update patricia.age = 14
= 14 (0x000E)

> update patricia.name[3] = char '\0'
= 0x00

> print patricia
 {
   name   = "pat"
   age  = 14 (0x000E)
   height  = 1.9
   spouse  = 0 (0x00000000)
   mother  = 29656 (0x000073D8)
   father  = 29712 (0x00007410)
   children  = 0 (0x00000000)
   child_1  = 0 (0x00000000)
   child_2  = 0 (0x00000000)
 }
>
```

Note that only variables that would normally be 'visible' to the program from its current location can be printed or updated.

The next section points out some of the complexities when combining user breakpoints with the virtual breakpoints used to support the single step functions.

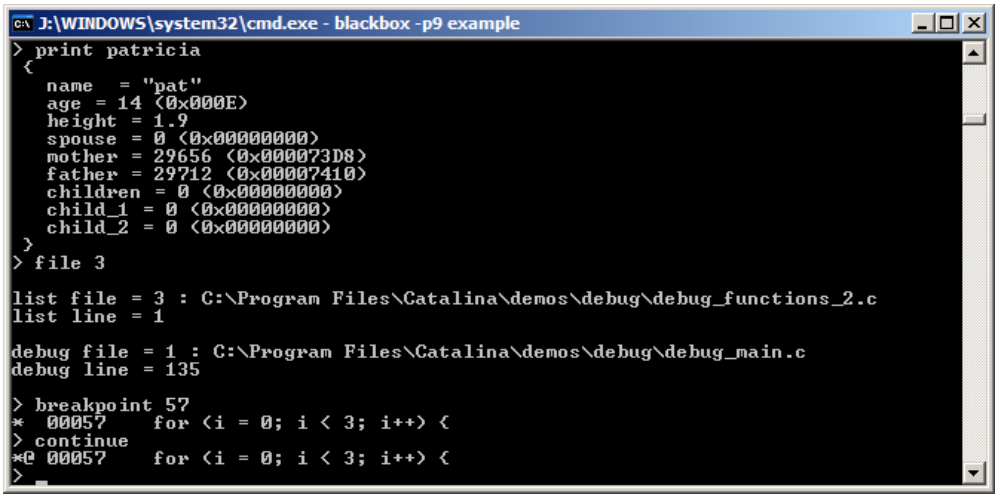
Combining User and Virtual Breakpoints

While working through this example, we have used both *user* breakpoints (which we set and cleared on specific lines) and *virtual* breakpoints (via the single step operations). However, care must be taken when combining the two different types of breakpoint, as the results can be unexpected.

To see this, select file 3 and put a breakpoint on line 57 (which is in function test_14). Then continue the program to reach that line. The commands to do this are:

```
file 3
breakpoint 57
continue
```

The result will look something like:



You may expect that you can now use the **Step Out** command to go back to line 167 in **main** (which is the line after **test_14** was called). However, this is not the case. Try it if you like – enter the command **out**. The program will instead step to the first line of **test_15**.

To understand why this is so (and why BlackBox is a bit 'unorthodox' in this respect) you need to understand what BlackBox is doing with virtual breakpoints. The rules that apply are as follows:

Step Next adds a virtual breakpoint to every line in the current function and removes virtual breakpoints from the first line of every function in the program.

Step Into adds a virtual breakpoint to the first line of every function in the program.

Step Out removes the virtual breakpoints from every line in the current function, and adds a breakpoint to the first line of every function.

This logic means that everything works as expected when using *only* virtual or *only* user breakpoints – but when the two are used in the same program, it is possible to enter a function *without* having set the virtual breakpoints that mean BlackBox will return to the calling function (i.e. the virtual breakpoints on each line of the calling function). In summary:

1. **Step Next** will always work as expected, even after a user breakpoint.
2. **Step Into** will not work as expected after a user breakpoint. If the current function does not have virtual breakpoints on each line, use **Step Next** before using **Step Into**.

3. **Step Out** will not work as expected after user breakpoint. Instead, put a manual breakpoint after the calling instead of relying on **Step Out**. Or use **Step External** or **Step Up**.

What to do next?

Some of the commands we have used in this tutorial have other options and uses. There are also other BlackBox commands we have not used– you can list them in BlackBox by using the **Help** command (note that the results may scroll off the screen):

```
help
```

The **BlackBox Reference Manual** contains full details on all BlackBox commands.

You can try out the other commands using the example program, or use BlackBox on any of the other demo programs provided with Catalina (after compiling them with the **–g** or **–g3** options).

Known Issues with BlackBox

This beta release of BlackBox has a few known limitations:

- BlackBox cannot resolve symbols that have been defined using **#define** - this can be confusing because in a source listing such symbols often look like any other variables or functions.
- BlackBox does not fully implement absolute and relative addresses – addresses are sometimes displayed as absolute, and sometimes as relative. For example, displaying the address of a function in the debugger does not give the same result as taking the address of a function in the C program, assigning that address to a variable and displaying the variable in the debugger – this is because one is absolute and one is relative. Converting between the two requires knowledge of the current memory model in use by the program, the current relocation value and the current code segment offset.
- Linux automatically raises DTR whenever a serial port is opened – this may result in the Propeller that you are attempting to debug receiving a RESET signal whenever you start BlackBox. This problem does not occur when using the mouse port for debugging since that port has no connection to the Propeller reset line – but when using the normal serial port it may be necessary to temporarily disconnect the RESET line when using BlackBox. Some platforms provide a jumper for this purpose (e.g. the RS232 port on Morpheus). On other platforms this must be done by disconnecting the RES line on the Prop Plug before starting BlackBox. Note that this is not an issue under Windows, which does not automatically raise DTR when a port is opened.